Guia Didactica Actualizada del Proyecto TD3 para una Protesis Mioelectrica

Terminos, parametros, formulas y estado final con politica residual

Proyecto `EMG_Prosthesis_TD3`

26 de marzo de 2026

Resumen

Esta guia didactica actualiza la ficha conceptual del proyecto hasta su estado final. Explica, con un lenguaje mas pedagogico que el reporte tecnico, que problema resuelve el sistema, que significa cada termino importante, que formulas dominan el comportamiento del agente, que bug cambio la lectura de muchas corridas, como se interpreta el benchmark `Agent7250` y por que la politica residual `Agent1850` con $\alpha_{\text{res}} = 0.20$ se toma como mejor punto final actual. Tambien resume scripts, parametros, criterios de seleccion y nuevas metricas de diagnostico.

Índice

1. Para que sirve esta guia **3**

2. Que problema resuelve el proyecto **3**

3. Glosario esencial **3**

4. Por que se usa TD3 **5**

5. Estado y accion del proyecto **5**
- 5.1. Observacion markov52 . . . 5
- 5.2. Accion del actor . . . 5

6. La reward actual y por que esta bien planteada **5**

7. El bug del simulador explicado de forma sencilla **6**

8. Linea valida antes de la fase residual **6**

9. Que se intento despues de Agent7250 **6**
- 9.1. Baja exploracion . . . 6
- 9.2. Threshold-aware exploration . . . 7
- 9.3. Reward con saturacion . . . 7
- 9.4. Warm-start offline . . . 7
- 9.5. Interfaz de accion alineada . . . 7
- 9.6. Residual TD3 . . . 7

10. Que es una politica residual **7**

11. Parametros importantes del estado final **8**

12. Metricas y diagnosticos nuevos de la fase residual **8**

13. Barridos de alpha_res **8**

14. Agente final del proyecto **9**

15. Scripts y flujos importantes **11**

16. Idea final que debe quedar clara **11**

1. Para que sirve esta guia

Esta guia intenta responder preguntas practicas:

- Que hace el proyecto exactamente.
- Que significa cada bloque tecnico importante.
- Por que se usa TD3.
- Que se corrige con la reward actual.
- Que fue el bug del simulador.
- Que es una politica residual.
- Que significa alpha_res.
- Cual es hoy el mejor agente y por que.

2. Que problema resuelve el proyecto

El proyecto quiere controlar una protesis robotica de mano a partir de senales EMG. La salida deseada no es un gesto discreto, sino una respuesta continua: cuanto abrir o cerrar cada parte de la protesis en funcion de la actividad muscular del usuario. Esta idea esta alineada con el control mioelectrico moderno [1] y con el planteamiento base del proyecto [2].

En forma corta:

$$\text{EMG} \rightarrow \text{estado} \rightarrow \text{actor TD3} \rightarrow \text{accion} \rightarrow \text{simulador} \rightarrow \text{reward}.$$

3. Glosario esencial

EMG

Actividad electrica muscular. En el proyecto sirve como proxy de la intencion motora.

Features EMG

Transformaciones numericas de la EMG cruda. Aqui se usan 5 features por cada uno de 8 canales, para un total de 40 dimensiones.

Estado

Vector que ve el agente en cada paso. En la formulacion valida actual es `markov52`.

Actor

Red que produce la accion.

Criticos

Redes que estiman que tan buena es una accion en un estado. TD3 usa dos criticos [3].

Replay buffer

Memoria de experiencias pasadas usada para entrenar de forma mas estable.

PWM

Magnitud de mando aplicada a los motores. En este proyecto la accion del actor termina convertida a niveles efectivos de PWM.

Tracking MSE

Error cuadratico medio entre referencia y salida de la protesis. Es la metrica central del proyecto.

Tracking MAE

Error absoluto medio. Es mas interpretable, pero penaliza menos los errores grandes.

ActionL2

Magnitud media de la accion. Se interpreta como esfuerzo de control.

Saturation fraction

Fraccion del tiempo en que la accion efectiva queda cerca del borde del actuador.

DeltaActionL2

Mide cuanto cambia la accion de un paso al siguiente. Es un indicador de brusquedad.

Checkpoint

Agente guardado durante entrenamiento. El mejor agente no tiene por que ser el ultimo.

Auditoria de checkpoints

Proceso de evaluar varios checkpoints con simulaciones cortas y largas para seleccionar el mejor por metricas reales, no por reward final de entrenamiento.

ConditionA y ConditionB

Reglas de aceptacion frente al benchmark `Agent7250`. `ConditionA` exige mejora estricta. `ConditionB` permite perder muy poco tracking a cambio de bajar bastante saturacion y esfuerzo.

Politica base

En la fase final es `Agent7250`. Sobre ella se construye la correccion residual.

Politica residual

Politica que no reemplaza totalmente a la base, sino que aprende una correccion pequena.

alpha_res

Escala que multiplica la correccion residual. Controla cuanto puede apartarse el agente final de la accion base.

4. Por que se usa TD3

TD3 es apropiado porque este proyecto es de accion continua, no de clasificacion discreta. Su logica central es:

- dos criticos para reducir sobreestimacion;
- actualizacion mas lenta del actor;
- suavizado de la politica objetivo.

Eso lo vuelve una buena opcion para control continuo con ruido y estados complejos [3, 4, 5].

5. Estado y accion del proyecto

5.1. Observacion markov52

La observacion valida actual se puede resumir como:

$$s_t = \begin{bmatrix} \phi_t \\ \mathrm{enc}_t \\ \Delta\mathrm{enc}_t \\ \tilde{a}_{t-1} \end{bmatrix} \in \mathbb{R}^{52},$$

donde:

- $\phi_t \in \mathbb{R}^{40}$: features EMG;
- $\mathrm{enc}_t \in \mathbb{R}^4$: encoders normalizados;
- $\Delta\mathrm{enc}_t \in \mathbb{R}^4$: cambio de encoder;
- $\tilde{a}_{t-1} \in \mathbb{R}^4$: accion efectiva previa.

5.2. Accion del actor

El actor produce:

$$a_t \in [-1, 1]^4.$$

Pero eso no significa que la protesis reciba exactamente ese valor continuo. Antes de actuar, la accion se remapea y se cuantiza a niveles efectivos. Esa es una idea crucial del repo: una politica buena no se juzga solo por su accion cruda, sino por la accion efectiva que termina recibiendo el actuador.

6. La reward actual y por que esta bien planteada

La reward vigente es:

$$r_t = -\frac{1}{4}\sum_{i=1}^{4}\left(e_{t,i}^2\lambda_a\tilde{a}_{t,i}^2\lambda_{\Delta a}\left(\tilde{a}_{t,i} - \tilde{a}_{t-1,i}\right)^2\right).$$

Donde:

- $e_{t,i}$: error de tracking;
- λ_a: peso del esfuerzo de accion;
- $\lambda_{\Delta a}$: peso de la suavidad.

La idea pedagogica es:

- el tracking manda;
- el esfuerzo se regulariza;
- la brusquedad tambien se regulariza.

Esto es mucho mas interpretable que la reward heredada inicial y evita parte de los problemas clasicos de *reward shaping* mal alineado [6, 7].

7. El bug del simulador explicado de forma sencilla

Una parte importante del proyecto consistio en descubrir que el simulador no estaba moviendo de verdad a la protesis en ciertas corridas antiguas. En otras palabras:

- el agente "entrenaba",
- la reward cambiaba,
- pero la planta no reaccionaba correctamente.

Eso fue grave porque invalidaba comparaciones numericas. Despues del fix, el proyecto entro en su fase valida. Esta es la razon por la que el benchmark serio no se toma de las corridas antiguas, sino de las corridas post-fix.

8. Linea valida antes de la fase residual

Cuadro 1: Linea post-fix principal antes de la mejora residual

Etapa	trackingMSE	trackingMAE	actionL2	saturationFraction	deltaActi
Baseline post-fix de 2000 episodios	0.052805	0.179093	0.674957	0.474999	0.63
`Agent7250` en 8000 episodios	0.043045	0.160336	0.596444	0.392086	0.32
`Agent11900` en 12000 episodios	0.045189	0.164639	0.649868	0.511786	0.30

Leccion didactica:

- mas episodios no garantizan una politica mejor;
- el mejor checkpoint debe elegirse por auditoria;
- un mejor reward final de entrenamiento no basta para declarar victoria.

9. Que se intento despues de Agent7250

Despues del benchmark `Agent7250` se probaron varias ideas porque el tracking ya era bueno, pero el control seguia siendo algo agresivo.

9.1. Baja exploracion

Se bajo el ruido de exploracion buscando una politica mas suave. Mejoro esfuerzo y saturacion, pero el tracking cayo demasiado.

9.2. Threshold-aware exploration

Se exploto el hecho de que el actuador tiene una discontinuidad real entre umbral y primer bin efectivo. Sirvio para entender mejor el repo, pero no produjo un agente nuevo mejor.

9.3. Reward con saturacion

Busco penalizar explicitamente cercania al borde. Resultado: politicas demasiado timidas.

9.4. Warm-start offline

Introdujo un prior mas suave desde datos sinteticos. Resultado: politicas demasiado conservadoras.

9.5. Interfaz de accion alineada

Intento hacer coincidir mejor la salida continua del actor con los niveles realmente utiles del actuador. Resultado: politicas demasiado agresivas.

9.6. Residual TD3

No intento reemplazar la politica base, sino corregirla. Esta fue la unica linea que produjo una mejora real.

10. Que es una politica residual

La politica residual final del proyecto se escribe como:

$$a(s) = \mathrm{clip}\Big(a_{\mathrm{base}}(s) + \alpha_{\mathrm{res}}\, a_{\mathrm{res}}(s, a_{\mathrm{base}}), -1, 1\Big).$$

Interpretacion sencilla:

- a_{base}: lo que haria `Agent7250`;
- a_{res}: una correccion aprendida;
- α_{res}: cuanto poder se le da a esa correccion.

Si α_{res} es pequeno, la residual apenas corrige. Si es grande, la residual puede separarse mucho de la base. El proyecto encontro que el mejor equilibrio estaba en $\alpha_{\mathrm{res}} = 0.20$.

11. Parametros importantes del estado final

Cuadro 2: Parametros clave del estado final del proyecto

Parametro	Valor	Que significa
`observationVariant`	`markov52`	Estado valido actual.
`rewardType`	`trackingMseActionRateReward`	Reward del proyecto.
`agent_id`	`td3_residual7250`	Familia final del agente.
`td3.useRecurrent`	`false`	La linea final es feedforward, no LSTM.
`td3Residual.baseCheckpointPath`	`Agent7250`	Politica base congelada.
`td3Residual.residualScale`	`0.20`	Mejor escala residual encontrada.
`td3Residual.hiddenUnits`	`32`	Anchura de la rama residual.
`explorationStd`	`0.02`	Ruido de exploracion residual.
`explorationStdMin`	`0.002`	Piso del ruido residual.
`explorationStdDecayRate`	`1e-4`	Decaimiento del ruido.
`rewardActionWeight`	`0.01`	Penalizacion de magnitud de accion.
`rewardDeltaActionWeight`	`0.05`	Penalizacion de cambio de accion.
`trainingEpisodes`	`2000`	Longitud de la corrida residual principal.
`trainingSaveAgentEvery`	`50`	Frecuencia de checkpoints.

12. Metricas y diagnosticos nuevos de la fase residual

La fase residual anadio diagnosticos utiles:

- `baseActionLog`: que habria hecho la politica base;
- `residualActionLog`: cuanto corrige la residual;
- `ResidualActionL2Mean`: magnitud media de la correccion;
- `BaseToFinalActionDeltaMean`: cuanto se separa la accion final de la base;
- `SignOverrideFractionMean`: cuantas veces la residual cambia el signo de la accion base.

Estos terminos ayudan a responder una pregunta importante: la residual esta corrigiendo con moderacion o esta intentando reemplazar por completo a la politica base.

13. Barridos de alpha_res

Hubo dos barridos:

- sweep corto: $\{0.10, 0.20, 0.30\}$,
- refinamiento fino: $\{0.18, 0.20, 0.22, 0.25\}$.

Resultado didactico:

- valores bajos no corrigieron bien;
- valores altos compraron mejora local de tracking con demasiado borde o esfuerzo;
- $\alpha_{\text{res}} = 0.20$ fue el punto util.

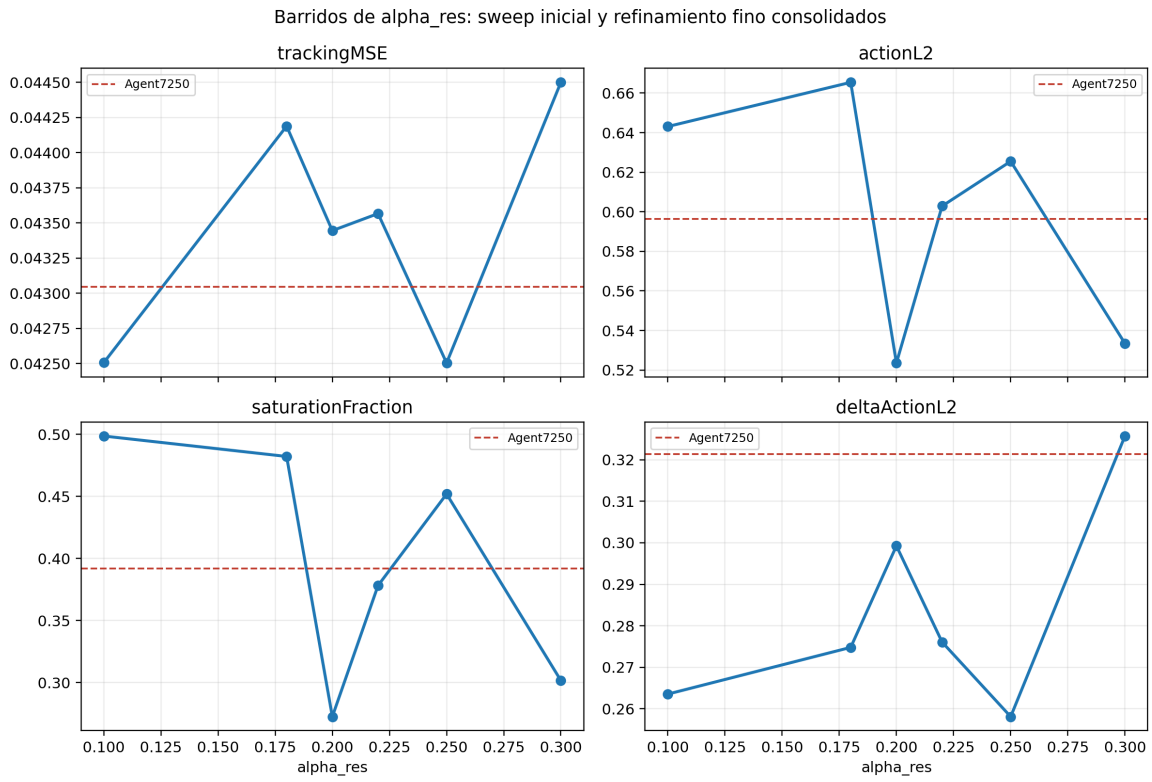

Figura 1: Barridos de alpha_res respecto al benchmark.

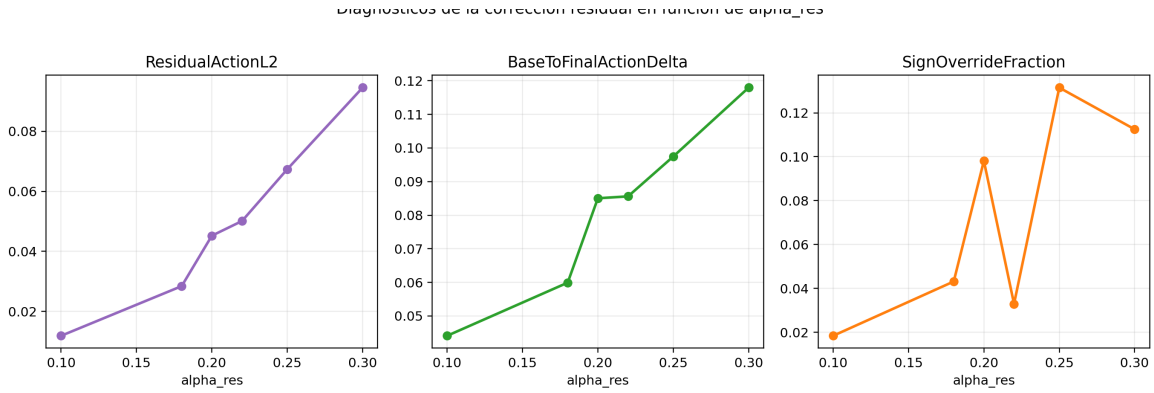

Figura 2: Diagnosticos de la correccion residual. La zona de alpha=0.20 mantiene una correccion util y moderada.

14. Agente final del proyecto

El mejor candidato final actual es:

- base congelada: `Agent7250`,
- mejora final: residual `Agent1850` con $\alpha_{\text{res}} = 0.20$.

Cuadro 3: Comparacion final entre benchmark y mejor residual

Metrica	`Agent7250`	Residual `Agent1850`
`trackingMSE`	0.043045	0.043445
`trackingMAE`	0.160336	0.163542
`actionL2`	0.596444	0.523597
`saturationFraction`	0.392086	0.272637
`deltaActionL2`	0.321385	0.299262

Interpretacion corta:

- el tracking casi no empeora;
- la saturacion baja mucho;
- el esfuerzo de control baja;
- por eso el resultado final entra en `ConditionB`.

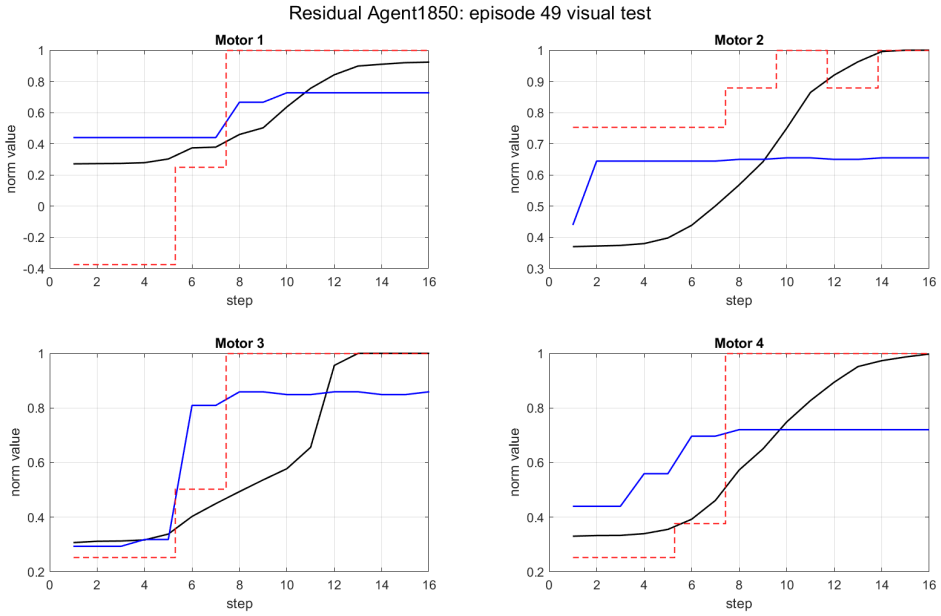

Figura 3: Ejemplo del test visual del residual `Agent1850`.

15. Scripts y flujos importantes

Cuadro 4: Scripts importantes del estado final

Script	Para que sirve
`trainInterface('td3',",")`	Flujo general de entrenamiento y test del baseline TD3.
`runCheckpointAudit(...)`	Audita checkpoints con fase rapida y fase completa.
`runCheckpointTest(...)`	Ejecuta test visual y metricas agregadas.
`run_agent7250_residual_policy_pilot`	Lanza el piloto residual principal.
`run_agent7250_residual_continuation`	Continua desde el mejor residual.
`run_agent7250_residual_alpha_sweep`	Ejecuta el barrido corto de alpha.
`run_agent7250_residual_alpha_refinement_sweep`	Refina el barrido fino de alpha.

16. Idea final que debe quedar clara

La leccion didactica mas importante del proyecto es esta:

> la mejora final no vino de volver a cambiar reward, ni de meter mas episodios sin criterio, ni de reconstruir todo desde cero; vino de partir de una politica base fuerte y aprender una correccion residual pequena, bien medida y auditada.

Ese es el punto final conceptual y operativo del proyecto actual.

Referencias

[1] Ziming Chen, Huasong Min, Dong Wang, Ziwei Xia, Fuchun Sun, and Bin Fang. A review of myoelectric control for prosthetic hand manipulation. *Biomimetics*, 8(3):328, 2023.

[2] Jose Gabriel Fuertes Bustos, Carlos Ayala, Lorena Isabel Barona, Marco E. Benalcazar, and Angel Leonardo Valdivieso. Continuous operation of a myoelectric hand prosthesis using the td3 reinforcement learning algorithm. Technical manuscript provided by the user, 2026. Baseline conceptual article used as historical reference for the project.

[3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. https://arxiv.org/abs/1802.09477, 2018. TD3 original paper.

[4] MathWorks. rltd3agent. https://www.mathworks.com/help/reinforcement-learning/ref/rl.agent.rltd3agent.html, 2026. Accessed: 2026-03-12.

[5] MathWorks. rltd3agentoptions. https://www.mathworks.com/help/reinforcement-learning/ref/rl.option.rltd3agentoptions.html, 2026. Accessed: 2026-03-12.

[6] Zelin Deng, Yunlong Dong, and Xing Liu. Reward shaping in reinforcement learning for robotic hand manipulation. *Neurocomputing*, 638:130204, 2025.

[7] I Lee, Hoang-Giang Cao, Cong-Tinh Dao, Yu-Cheng Chen, and I-Chen Wu. Gradient-based regularization for action smoothness in robotic control with reinforcement learning. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 603–610, 2024.